

2026-2 INHATC

피지컬 AI 응용

5주차 · PyTorch - 텐서와 학습 루프

김태령

복습 · 재현 퀴즈

KMeans · PCA 세 줄 재현, 그리고 PyTorch 3주 로드맵

PART 1 · 왜 PyTorch 인가

Keras 와 무엇이 같고 무엇이 다른가

PART 2 · 텐서

numpy 배열과 무엇이 다른가 · shape · dtype · 왕복

PART 3 · 자동미분

requires_grad → backward → grad · 경사하강한 줄

PART 4 · 학습 루프 다섯 줄

3주차 회귀를 PyTorch 로 다시 · 손실 곡선 · 층 하나 더

정리 · 과제

오늘의 목표 - Keras 의 fit() 한 줄 안에서 일어나는 일을 PyTorch 다섯 줄로 직접 써 보기



재현 · 곡선

재현 - 빈 화면에

KMeans 와 PCA, 각각 세 줄

```
Xs = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=2, random_state=42).fit(Xs)
km.labels_
pca = PCA(n_components=2).fit(Xs)
Z = pca.transform(Xs)
pca.explained_variance_ratio_
```

- 공통점 - 스케일을 먼저 맞춘다.

남긴 곡선

max_depth 를 돌렸을 때

- 훈련 점수는 올라가고, 시험 점수는 꺾인다 - 과적합
- 손잡이(하이퍼파라미터)는 곡선의 꼭대기에서 멈춘다
- 신경망 - 학습률(lr)과 반복 횟수(epoch)

PART 1

왜 PyTorch 인가

From Keras to PyTorch

3주 로드맵 · 요즘 로봇 AI 는 무엇으로 만드나 · Keras 대응표 · 수동변속을 배우는 이유

3주 Pytorch

1

5주차 · 텐서와 학습 루프

텐서 · 자동미분 · 다섯 줄 루프 · 3주차 회귀 재현

2

6주차 · 분류와 MLP

고장 데이터 분류 · 미니배치 · 학습 곡선 · MNIST 손글씨

3

7주차 · CNN 과 사전학습

합성곱 · 이미지 분류 · 사전학습 모델 가져다 쓰기

4

11주차~ · 비전 · VLA

같은 루프, 더 큰 모델 - 로봇이 보고 판단하는 법

- 세 주 내내 코드의 골격은 하나. 다섯 줄 루프가 CNN 에서도, 11주차 VLA 에서도 그대로
- 3주차 5단계(나누고·만들고·학습하고·예측하고·평가한다)도 그대로. 바뀌는 것은 ‘학습하고’ 한 칸이 다섯 줄로.
- 지금까지 `fit()` 이 대신해 주던 일을 옮겨 오기

요즘 로봇 AI 는 무엇으로 만들어지나

VLA 모델은 거의 전부 PyTorch

OpenVLA · $\pi 0$ · SmoVLA · GR00T - 2주차
에 본 로봇 파운데이션 모델들의 공개 코드는 모두
PyTorch 로

허깅페이스 생태계

사전학습 모델을 내려받아 미세조정하는 표준 통로.
transformers · LeRobot 라이브러리가
PyTorch 위에

연구 → 현장

논문 코드가 PyTorch 로 나오고, 그대로 학습·배포
까지 이어집니다. 새 모델이 나오면 가장 먼저 열어 볼
수 있는 도구.

Keras 와의 비교

Keras 는 ‘빨리 만들기’에 좋고, PyTorch 는 ‘안을 열어 고치기’에 좋다.
남이 만든 로봇 모델을 가져다 우리 문제에 맞추려면 안을 열 줄 알아야 한다.

Keras ↔ PyTorch 대응표

Keras

세 줄이면 학습

```
model = Sequential([Dense(1, input_shape=(2,))])
model.compile(optimizer='sgd', loss='mse')
model.fit(X, y, epochs=100)
model.predict(X_new)
```

- **fit() 한 줄 안에서 무슨 일이 일어나는지는 보이지 않음**

오늘 · PyTorch

같은 일을 펼쳐서

```
model = nn.Sequential(nn.Linear(2, 1))
loss_fn = nn.MSELoss()
opt = optim.SGD(model.parameters(), lr=0.1)
for epoch in range(100): # ← fit() 을 다섯 줄로
    opt.zero_grad()
    pred = model(X); loss = loss_fn(pred, y)
    loss.backward(); opt.step()
model(X_new)
```

- **compile → 부품 두 개 · fit → 다섯 줄 루프. 나머지는 이름만 다름**

왜 굳이 펼쳐 쓰나

Keras · 자동변속

fit() 이 기어를 알아서 바꿈. 빠르고 편하지만, 이상하게 굴 때 어디를 만져야 할지 모름

PyTorch · 수동변속

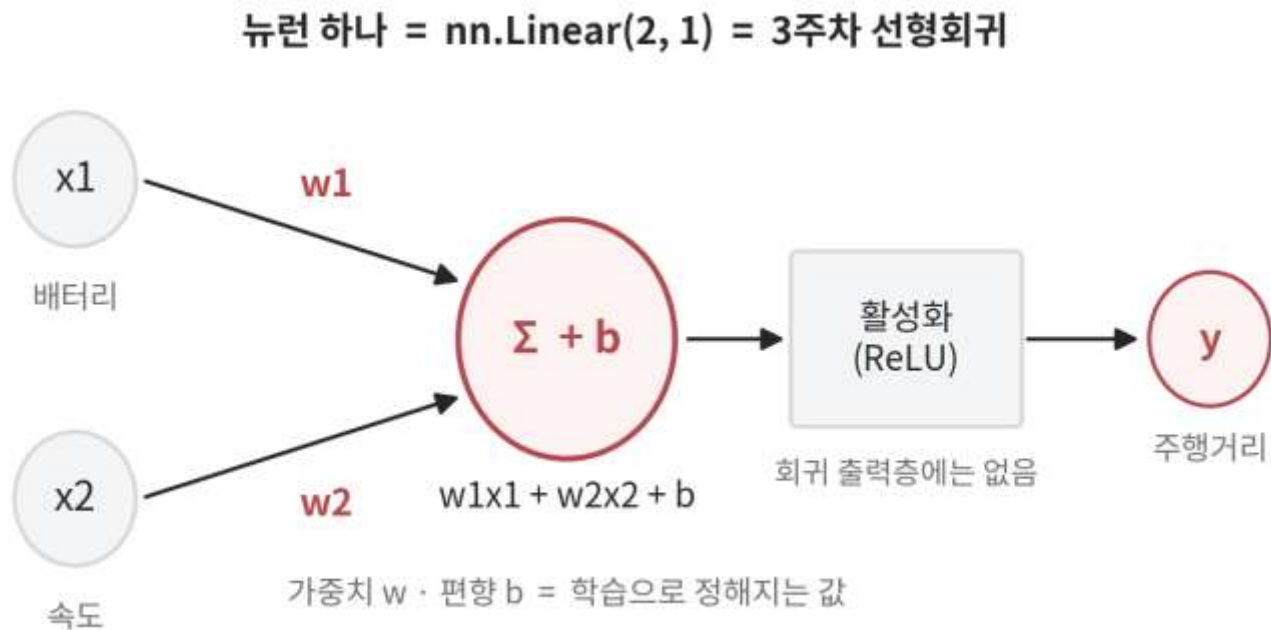
기어를 손으로 바꿈. 처음엔 번거롭지만, 손실이 왜 안 내려가는지 매 줄에서 볼 수 있음

실무에서 실제로 하는 일

남의 모델을 열어 마지막 층만 바꾸기, 손실 함수 바꾸기, 입력 두 개 넣기 - 루프 안을 만지는 일

- 오늘 외울 것 (학습 루프 다섯 줄). 나머지(텐서·자동미분)는 다섯 줄을 이해하기 위한 준비
- 다섯 줄: `opt.zero_grad()` → `pred = model(X)` → `loss = loss_fn(pred, y)` → `loss.backward()` → `opt.step()`

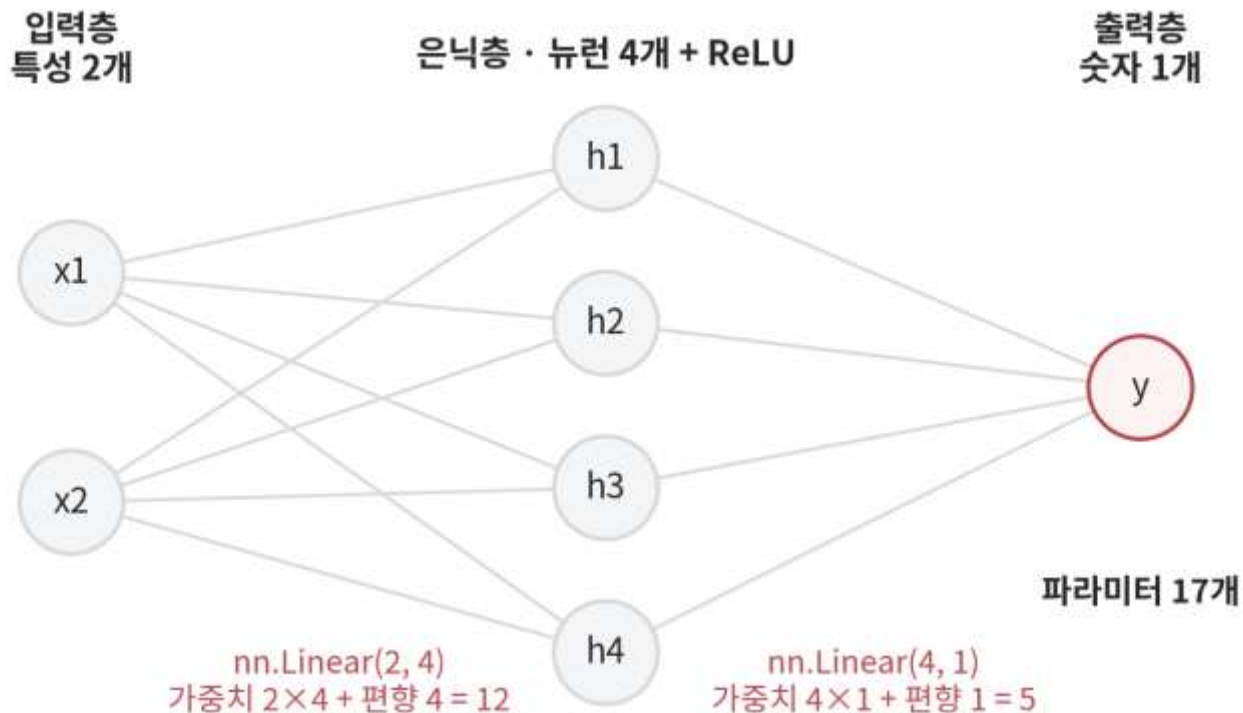
뉴런 하나 = 선형회귀 + 꺾기



- 입력 $\times$ 가중치, 다 더하고, 편향을 더한다 - 3주차 선형회귀 그대로
- 가중치 w 와 편향 b 가 학습으로 정해지는 값(파라미터)
- 활성화 함수로 한 번 꺾는다 - 이게 있어야 직선이 아닌 것도 배운다
- `nn.Linear(2, 1)` 이 이 그림 한 개. 파라미터 3개.

용어 - 뉴런(노드) · 가중치 weight · 편향 bias · 활성화 activation · 파라미터 = 학습되는 값 전부

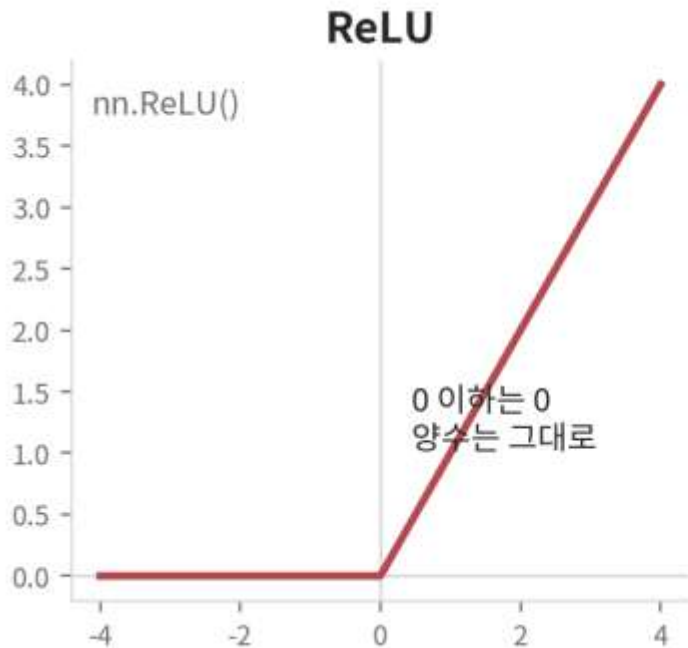
층을 쌓으면 - 입력층 · 은닉층 · 출력층



- 층 = 뉴런 여러 개를 나란히. 앞 층의 출력이 다음 층의 입력
- 은닉층이 하나라도 있으면 신경망(MLP). 뉴런 수 · 층 수는 하이퍼파라미터
- 파라미터 수 = (입력 × 출력 + 출력) 을 층마다 더한다
- 실습 ④ 는 은닉 16개 → $2 \times 16 + 16 + 16 \times 1 + 1 = 65$ 개

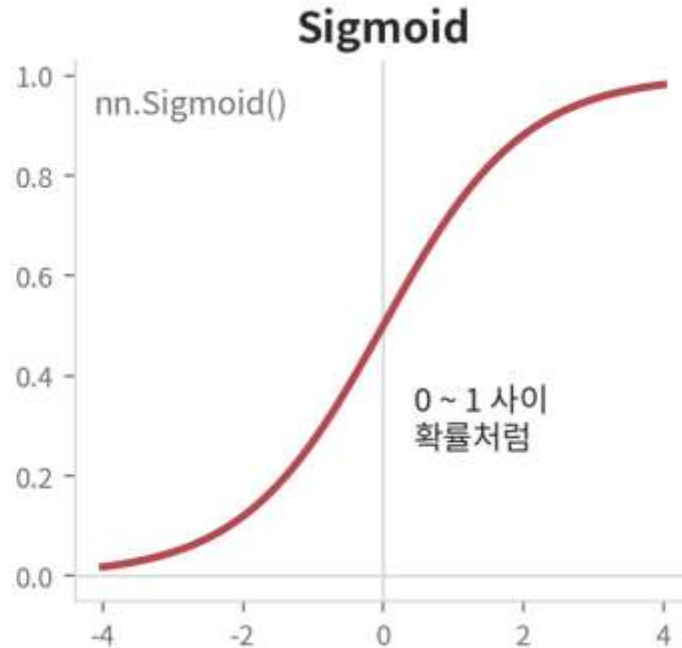
```
nn.Sequential( nn.Linear(2,16), nn.ReLU(),  
nn.Linear(16,1) )
```

활성화 함수 - 왜 꺾어야 하나



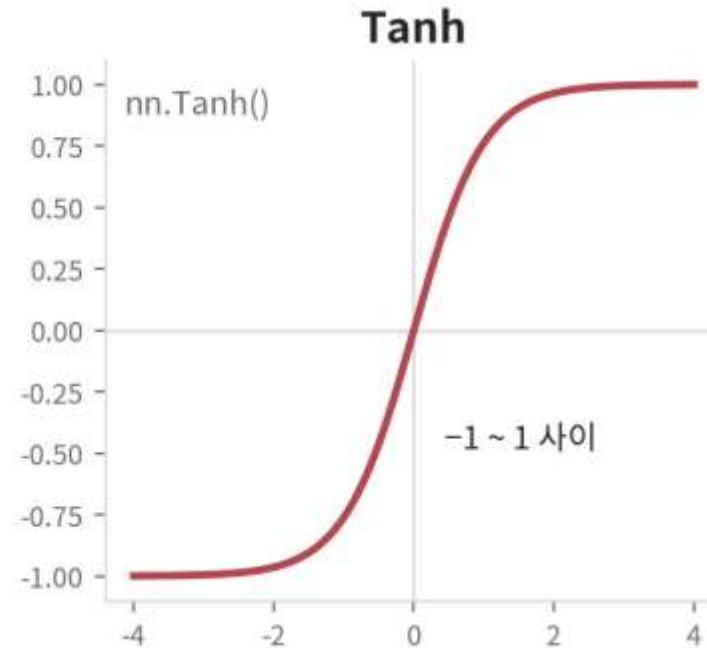
직선을 아무리 쌓아도 직선

Linear → Linear 는 결국 하나의 Linear. 꺾는 함수가 사이에 있어야 곡선을 배운다.



은닉층엔 ReLU

계산이 싸고 잘 된다. 요즘 기본값. 실습 ④ 에서 씀.



출력층은 문제가 정한다

회귀: 없음(숫자 그대로) · 이진분류: Sigmoid(확률) · 다중
분류: Softmax - 6주차

PART 2

텐서

Tensor

numpy 배열 + 두 가지 · 만들기 · shape 와 dtype · numpy 왕복 · GPU 한 줄

텐서 = numpy 배열 + 두 가지

numpy 배열과 같은 것

숫자 상자. shape 가 있고, $+$ $-$ $\times$ $\div$ 와 행렬곱이 되고, 인덱싱·reshape 이 됨. 1주차에 배운 그대로.
`torch.tensor([[1., 2.], [3., 4.]])`

+ ① GPU 로 갈 수 있다

같은 코드가 `.to('cuda')` 한 줄로 GPU 에서 진행됨
`.to(device)`

+ ② 기울기를 기억한다

이 값이 어떤 계산을 거쳐 나왔는지 기록해 두었다가, 나중에 미분을 자동으로
`requires_grad=True`

```
import torch
t = torch.tensor([[1., 2., 3.], [4., 5., 6.]]) # 소수점 → float32 (신경망의 기본 dtype)
t.shape          # torch.Size([2, 3])        ← numpy 의 .shape 와 같은 뜻
t.dtype          # torch.float32
t.sum(), t.mean() # tensor(21.), tensor(3.5000)
t * 2, t.T, t @ torch.ones(3, 1) # 원소 곱 · 전치 · 행렬곱 - 전부 numpy 와 같다
```

numpy 와 왕복하기 · 그리고 dtype 함정 하나

```
import numpy as np
a = np.array([1, 2, 3])
t = torch.from_numpy(a)          # numpy → 텐서   dtype = torch.int64 ← 정수!
t = torch.tensor(a, dtype=torch.float32) # 신경망에 넣을 거면 float32 로
t.numpy()                       # 텐서 → numpy   (requires_grad 가 있으면 .detach().numpy())

# 3주차 데이터를 텐서로 - 오늘 실습에서 그대로 쓰는 두 줄
X_t = torch.tensor(X_train, dtype=torch.float32)          # (96, 2)
y_t = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1) # (96,) → (96, 1)
```

- dtype - 신경망의 가중치는 float32. 정수(int64)나 float64 데이터를 그냥 넣으면 'dtype 이 다르다'는 에러. 만들 때 dtype=torch.float32 를 습관으로.
- shape - y 는 (n,) 이 아니라 (n, 1). 모델 출력이 (n, 1) 이라 손실 계산 때 모양이 맞아야 함. reshape(-1, 1) 을 습관으로.

shape 읽는 법 - 0차원부터 4차원까지

shape 는 '대괄호를 몇 겹 벗겨야 숫자가 나오나' — 차원 수 = `len(t.shape)`

0차원 · 스칼라



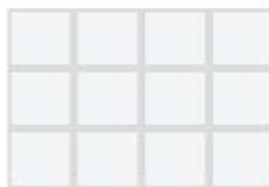
`shape []`
`torch.tensor(3.5)`
`loss.item()`

1차원 · 벡터



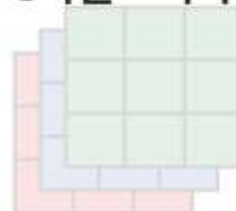
`shape [4]`
`y_train (96,)`
`→ reshape(-1,1)`

2차원 · 행렬



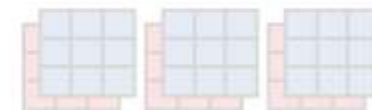
`shape [3, 4]`
`X_train (96, 2)`
행 = 샘플, 열 = 특성

3차원 · 이미지



`shape [C, H, W]`
`[3, 28, 28]`
채널 · 높이 · 너비

4차원 · 배치



`shape [N, C, H, W]`
`[32, 3, 28, 28]`
한 번에 넣는 32장 (7주차)

- 오늘 쓰는 것은 2차원까지 - X는 (샘플, 특성), y는 (샘플, 1)
- 7주차 이미지부터 3·4차원. 그때도 규칙은 같다: 맨 앞이 '몇 개', 나머지가 '하나의 모양'
- `t.shape` · `t.ndim` · `t.reshape(-1, 1)` · `t.unsqueeze(0)` - 모양을 확인하고 바꾸는 네 가지

CPU-GPU 설정

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(device)          # Colab 런타임 → GPU 를 켜면 'cuda', 아니면 'cpu'

model = model.to(device)    # 모델도, 데이터도 같은 곳에 있어야 한다
X_t = X_t.to(device)
y_t = y_t.to(device)
```

6주차

데이터가 작아서 CPU 로 충분
device 줄은 써 두기만

CNN 부터

이미지 수만 장을 돌리면 CPU 는 수십 분, GPU 는 수십 초.
런타임 → GPU 로 변경

자주 나는 에러

모델은 cuda, 데이터는 cpu - '같은 device 에 있어야 한다'는 에러. 둘 다 .to(device).
Expected all tensors on the same device

실습 ① 텐서 다루기

해 볼 것

노트북 실습 ① 셀을 채워 실행

- 1) `torch.arange(6)` 을 (2, 3) 으로 reshape 하고 shape 출력
- 2) 3주차 회귀 데이터 `X_train, y_train` 을 float32 텐서로 (`y` 는 (n, 1))
- 3) `X_t` 의 열별 평균 `X_t.mean(dim=0)` 을 출력 - numpy 의 `axis=0` 과 같은 뜻
- 4) `y_t` 를 다시 numpy 로 되돌려 앞 3개 출력
- **예상 출력이 노트북에 - 그대로 나오면 통과**

확인 질문

다 한 학생은 옆 사람에게 설명

- `torch.from_numpy(np.array([1, 2, 3]))` 의 dtype 은?
- `y_t.shape` 가 (96,) 이면 무엇이 문제인가?
- numpy 에서 axis 라고 부르던 것을 torch 에서는 _ _ _ 라고 부른다
- **답: int64 / 손실 계산 때 (96,1) 과 안 맞음 / dim**

PART 3

자동미분

Autograd

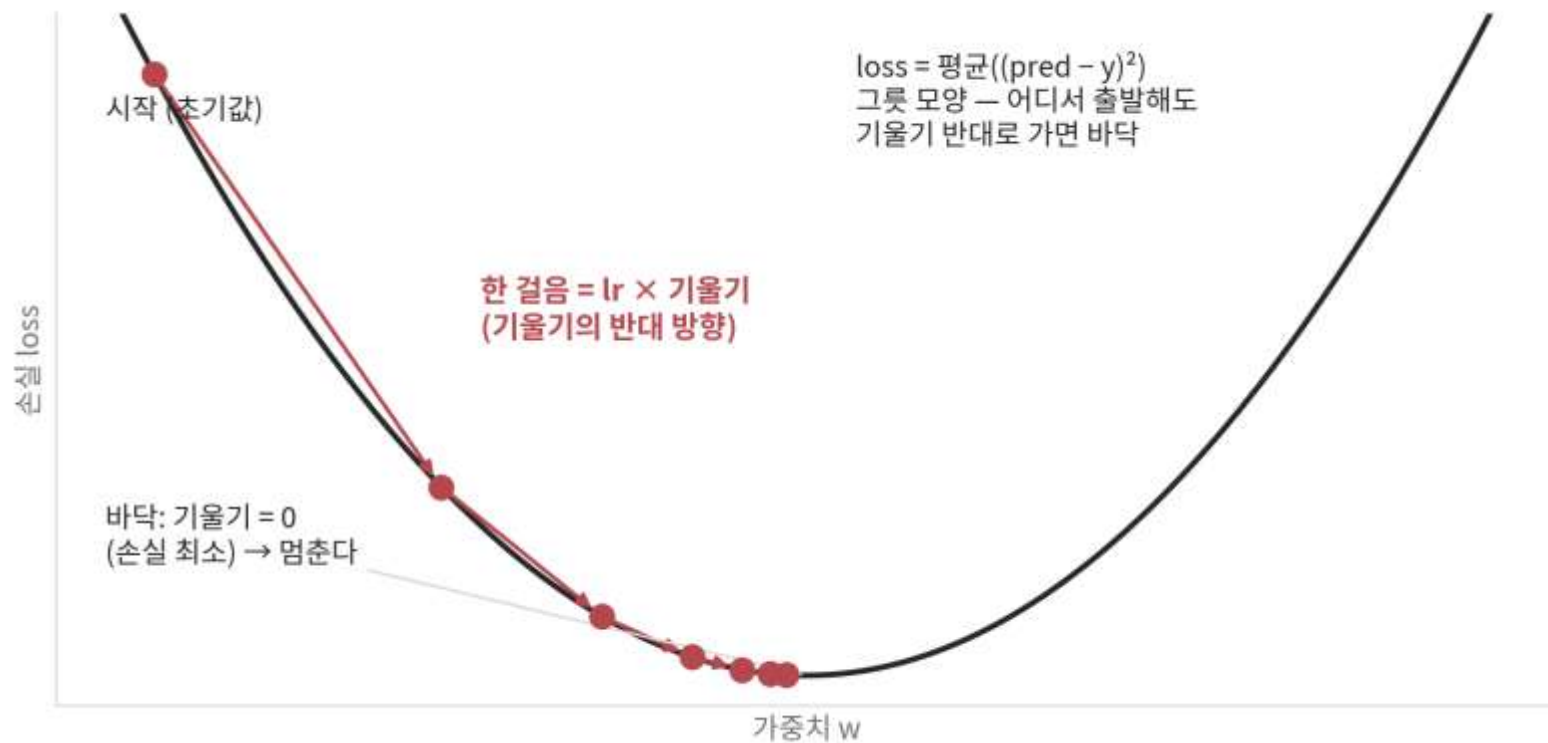
학습은 손실을 줄이는 방향으로 조금씩 · 기울기를 자동으로 · 경사하강 한 줄 · 학습률

학습이란 - 손실이 줄어드는 방향으로 w 를 조금씩 옮기는 것



- 3주차의 `LinearRegression().fit()` 은 이 과정을 공식 한 방으로. 신경망은 공식이 없어서 한 걸음씩
- 안개 낀 산에서 내려오기: 발밑의 기울기만 보고 낮은 쪽으로 한 발. 그 '발밑 기울기'를 구해 주는 것이 자동미분(`autograd`)
- 한 발의 크기가 학습률(`lr`). 너무 작을때와 클 때 둘다.
- **PyTorch** 가 우리 대신 해 주는 일, 기울기 계산 뿐이고 나머지는 우리가 함

손실 함수는 그릇 - 기울기 반대로 내려간다



- 손실 = 틀린 정도를 숫자 하나로. 회귀는 MSE
- w 를 바꾸면 손실이 바뀐다 - 그 관계가 그릇 모양
- 기울기 = 지금 위치에서 그릇이 기운 방향과 정도
- 경사하강 = 기울기의 반대로 lr 만큼 한 걸음, 반복

$$w = w - lr \times grad$$

- 바닥에선 기울기 0 $\rightarrow$ 더 안 움직인다 = 학습 끝

기울기를 자동으로

```
x = torch.tensor(3.0, requires_grad=True) # ① '이 값에 대한 기울기를 기억해 달라'
y = x**2 + 2*x                             # 계산 - PyTorch 가 과정을 기록한다
y.backward()                               # ② 기록을 거꾸로 따라가며 미분
x.grad                                     # ③ tensor(8.) ← dy/dx = 2x + 2 = 8

# 벡터도 똑같다
w = torch.tensor([1.0, 2.0], requires_grad=True)
z = (w**2).sum(); z.backward()
w.grad                                     # tensor([2., 4.]) ← 각 원소에 대해 2w
```

requires_grad=True

‘학습할 값’ 표시. 모델의 가중치는 nn.Linear가 알아서

.backward()

손실에서 출발해 모든 가중치의 기울기를 한 번에 계산. 루프의 4번째 줄.

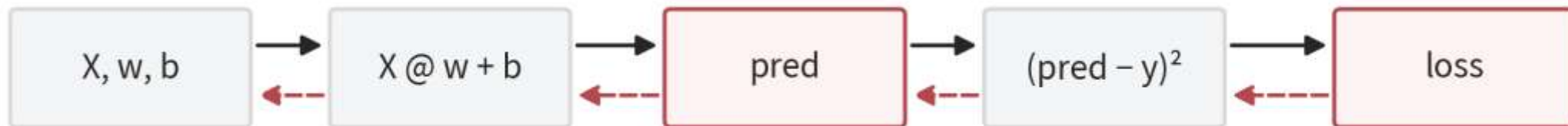
.grad 는 쌓인다

backward를 두 번 하면 기울기가 더해짐. 그래서 매번 zero_grad() - 루프의 1번째 줄.

순전파와 역전파 - 값은 앞으로, 기울기는 뒤로

순전파 forward — `pred = model(X); loss = loss_fn(pred, y)` 값을 앞으로

PyTorch 는 이 계산 과정을 그래프로 기록한다 (`requires_grad`)



역전파 backward — `loss.backward()` 기울기를 거꾸로: $d \text{ loss} / d \text{ pred} \rightarrow d \text{ loss} / d w$

`w.grad, b.grad` 에 저장 ← `opt.step()` 이 쓴다

순전파 forward

입력에서 손실까지 계산. 다섯 줄의 2·3번째 줄.

```
pred = model(X); loss = loss_fn(pred,  
y)
```

역전파 backward

손실에서 거꾸로 각 파라미터의 기울기를 계산. 4번째 줄.

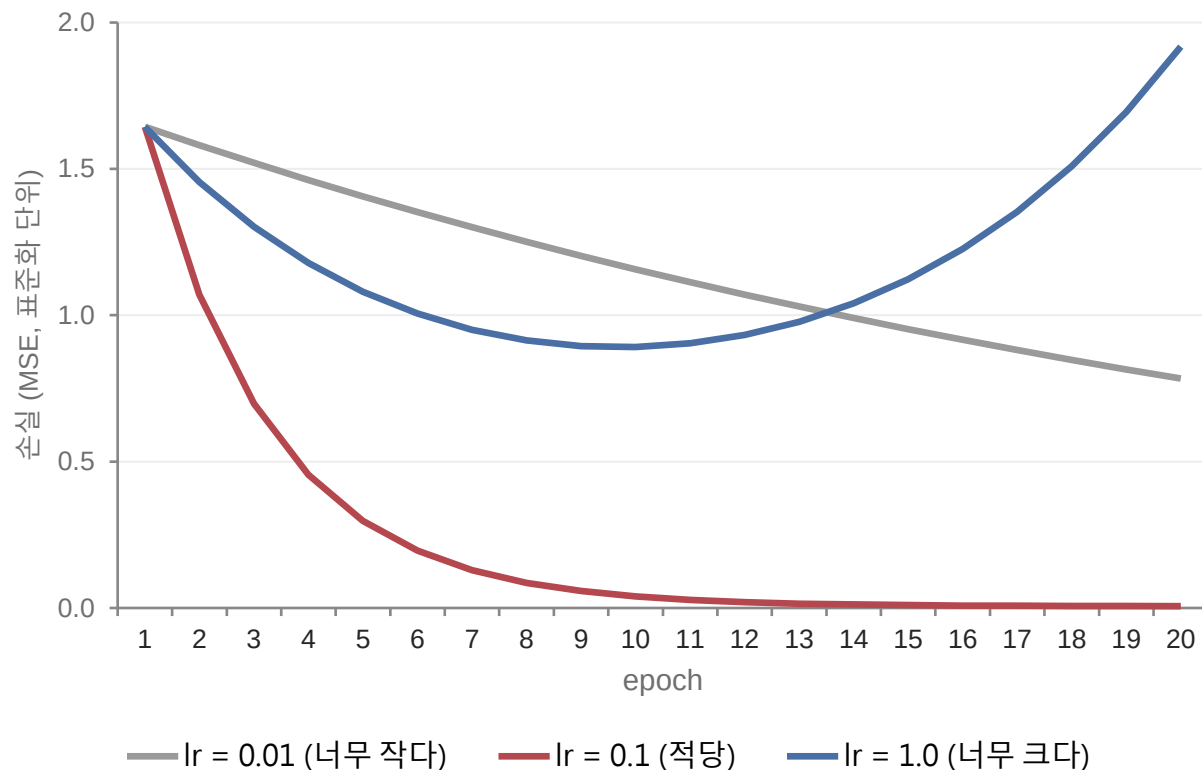
```
loss.backward() → w.grad
```

갱신 step

기울기로 파라미터를 한 걸음 옮긴다. 5번째 줄.

```
opt.step()
```

경사하강 한 줄 - 그리고 학습률이라는 손잡이



한 걸음 = 기울기 반대 방향으로 lr 만큼

```
with torch.no_grad():  
    w -= lr * w.grad  
    b -= lr * b.grad  
w.grad.zero_(); b.grad.zero_()
```

- lr 0.1 - 20번 만에 바닥. 0.01 - 같은 20번에 절반도 못 내려옴. 1.0 - 내려가다 다시 올라감(발산).
- 4주차 max_depth 곡선과 같은 종류의 손잡이. 정답은 없고, 돌려 보고 곡선을 봄
- 실무 기본값: SGD 0.01~0.1, Adam 0.001. 손실이 튀면 lr 을 10분의 1로.

용어 세 개 - epoch · 배치 · 학습률

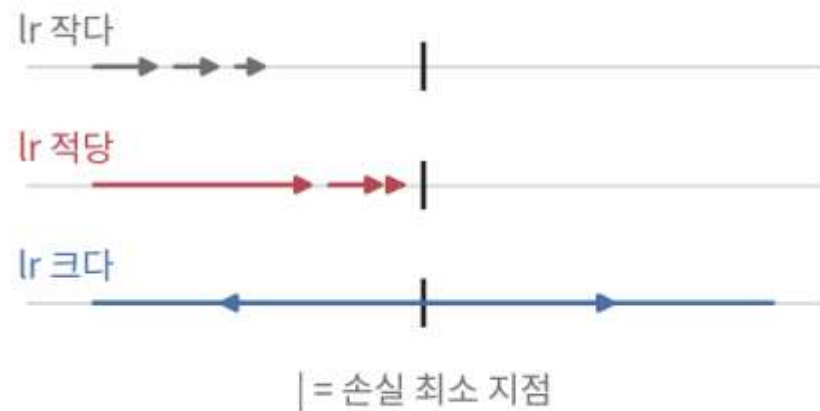
훈련 데이터 96개 (그림은 12개)



epoch 1 바퀴 = 배치 3개 = 다섯 줄 루프 3번 (opt.step 3번)

batch_size=32 · 오늘 실습은 96개를 한 번에 = 배치 1개 (6주차 DataLoader)

학습률 lr = 한 걸음의 크기



epoch

훈련 데이터 전체를 한 바퀴. for epoch in range(100) = 100바퀴.

너무 적으면 덜 배움 · 많으면 과적합

배치 batch

한 번에 넣는 묶음. 데이터가 크면 나눠 넣는다(6주차 DataLoader).

batch_size=32 가 흔한 기본값

학습률 lr

한 걸음의 크기. 손실이 튀면 10분의 1, 안 움직이면 10배.

SGD 0.01~0.1 · Adam 0.001

실습 ② 손으로 기울기

해 볼 것

노트북 실습 ② 셀

- 1) $x = 2.0$ 에서 $y = 3x^2$ 의 기울기를 autograd 로 구하기 (답: 12)
- 2) $w = [1, 2, 3]$ 에 대해 $z = (w^2).sum()$ 의 기울기 (답: [2, 4, 6])
- 3) `backward()` 를 두 번 연속 호출하면 `grad` 가 어떻게 되나 - 직접 확인
- 4) 3주차 회귀 데이터로 '`w -= lr * w.grad`' 를 30번 돌려 손실이 줄어드는지 출력
- 4)번 손실이 1.0 → 0.006 근처로 떨어지면 통과

확인 질문

- `requires_grad=True` 를 안 붙이면 `.grad` 는?
- `zero_grad()` 를 빼면 무슨 일이 생기나?
- `lr` 을 1.0 으로 바꾸면?
- 답: None / 기울기가 누적돼 이상한 방향으로 감 / 손실이 줄다가 다시 커짐(발산)

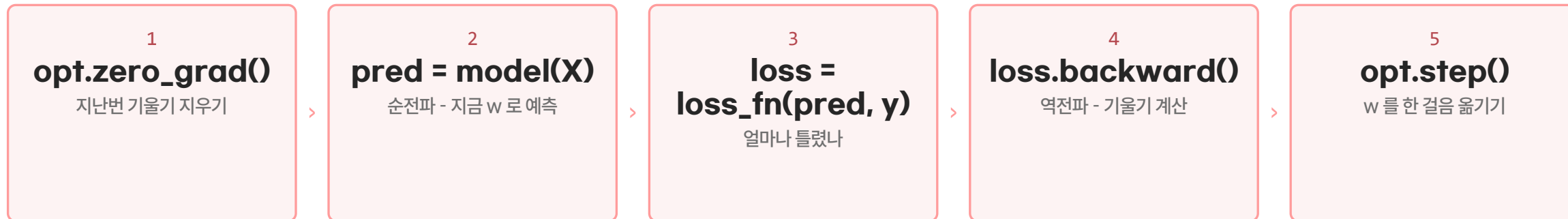
PART 4

학습 루프 다섯 줄

The Training Loop

부품 네 개 · 다섯 줄 · 3주차 회귀를 PyTorch 로 · 같은 답 · 총 하나 더 있기

오늘 외울 것 - 다섯 줄



```
for epoch in range(100):  
    opt.zero_grad()  
    pred = model(X_t)  
    loss = loss_fn(pred, y_t)  
    loss.backward()  
    opt.step()  
    if epoch % 10 == 0: print(epoch, loss.item())    # 손  
실이 내려가는지 눈으로
```

- PART 3 실습 ②의 4)번과 같은 일입니다. `w -= lr * w.grad`를 `opt.step()` 이, `grad.zero_()` 를 `opt.zero_grad()` 가 대신함
- 순서가 곧 의미.
지우고 → 예측하고 → 틀린 만큼 재고 → 기울기 구하고 → 옮긴다.
- Keras 의 `fit()` 이 감춰 두었던 것

루프 앞에 놓는 부품 네 개

① 모델 `nn.Sequential`

층을 순서대로 쌓은 것. `nn.Linear(2, 1)` 은 ‘입력 2개 → 출력 1개’ 선형층 = 3주차 선형회귀 그 자체.

```
model = nn.Sequential(nn.Linear(2, 1))
```

② 손실 `nn.MSELoss`

회귀는 MSE. 분류는 `CrossEntropyLoss`. `mean_squared_error` 와 같은 계산.

```
loss_fn = nn.MSELoss()
```

③ 옵티마이저 `optim.SGD`

‘w를 어떻게 옮길까’ 담당. `model.parameters()` 를 넘겨 주면 그 값들을 대신 갱신. Adam 은 발 크기를 알아서 조절하는 버전.

```
opt = optim.SGD(model.parameters(), lr=0.1)
```

④ 데이터 텐서

`float32`, `y` 는 `(n, 1)`. 그리고 표준화. 신경망은 스케일에 KMeans 보다 훨씬 민감

`X_t`, `y_t` (`StandardScaler` 거친 뒤)

Keras 로 말하면 - ① `Sequential` ② `compile(loss=...)` ③ `compile(optimizer=...)` ④ 똑같은. `compile` 한 줄이 ②③ 두 부품으로 나뉜 것뿐.

손실 함수와 옵티마이저 - 무엇을 고르나

	회귀 (오늘)	분류 (6주차)
손실 함수	<code>nn.MSELoss()</code>	<code>nn.CrossEntropyLoss()</code>
출력층	<code>nn.Linear(..., 1)</code>	<code>nn.Linear(..., 클래스 수)</code>
예측 읽기	숫자 그대로	<code>argmax</code> → 클래스 번호
3주차 대응	<code>mean_squared_error</code>	<code>accuracy_score</code> 로 평가

optim.SGD

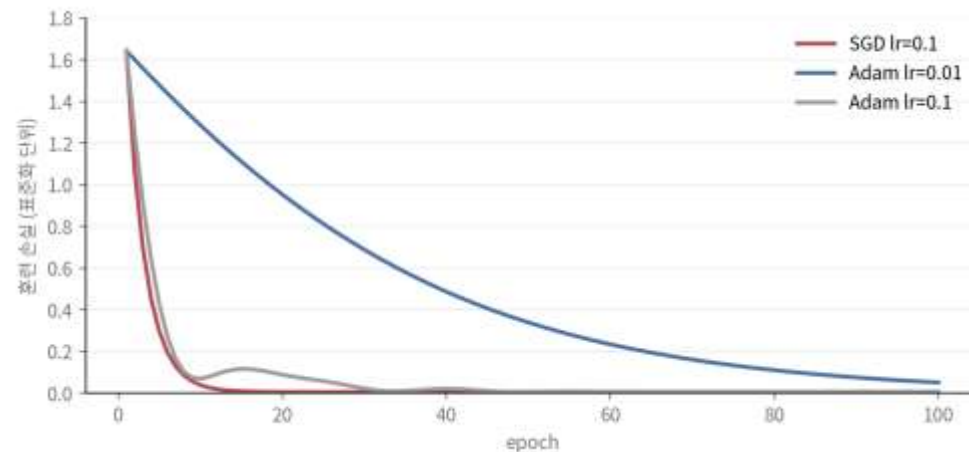
한 걸음 = $lr \times$ 기울기. 단순하고 투명. 오늘 실습.

$lr=0.1$ (표준화했을 때)

optim.Adam

걸음 크기를 파라미터마다 알아서 조절. 실무 기본값.

$lr=0.001$ 부터 시작

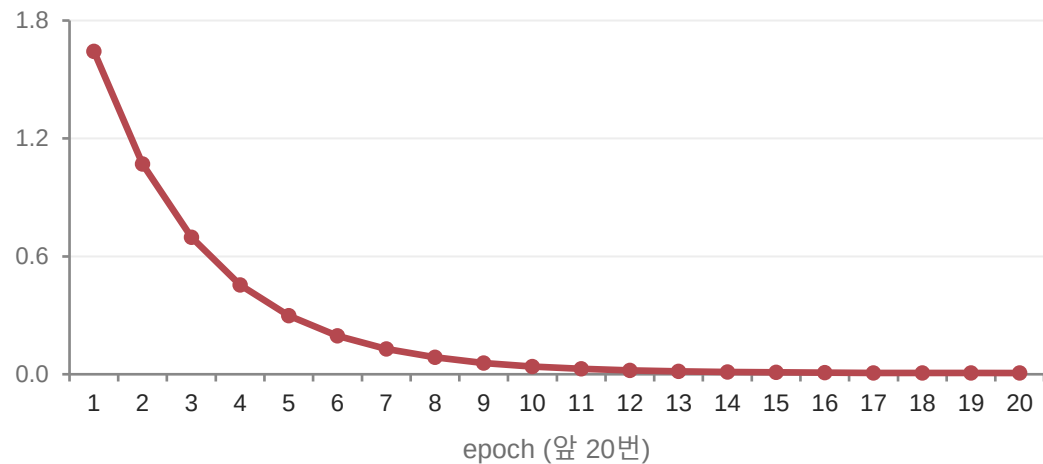


- 손실은 문제(회귀/분류)가 정하고, 옵티마이저는 취향 - 모르면 Adam.
- 그림: 같은 데이터에서 SGD 0.1 은 20 epoch 에 바닥, Adam 0.01 은 100 epoch 에도 0.05 - lr 이 작으면 Adam 도 느리다.
- 둘 다 다섯 줄은 그대로. 바뀌는 건 `opt = 한 줄`.

실습 ③ 회귀를 PyTorch 로 - 전체 코드

```
# 1 나누고 - 3주차와 완전히 같다
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 1.5 스케일 맞추고 - x 도 y 도 (신경망은 여기서 자주 죽는다)
sx = StandardScaler().fit(X_train); sy = StandardScaler().fit(y_train.reshape(-1, 1))
X_t = torch.tensor(sx.transform(X_train), dtype=torch.float32)
y_t = torch.tensor(sy.transform(y_train.reshape(-1, 1)), dtype=torch.float32)
# 2 만들고 - 부품 세 개
torch.manual_seed(0) # ← 3주차 random_state 와 같은 역할
model = nn.Sequential(nn.Linear(2, 1)); loss_fn = nn.MSELoss(); opt = optim.SGD(model.parameters(), lr=0.1)
# 3 학습하고 - 다섯 줄
for epoch in range(100):
    opt.zero_grad(); pred = model(X_t); loss = loss_fn(pred, y_t); loss.backward(); opt.step()
# 4 예측하고 5 평가한다 - 표준화를 되돌려서 3주차와 같은 단위(m)로
with torch.no_grad():
    pred_test = sy.inverse_transform(model(torch.tensor(sx.transform(X_test), dtype=torch.float32)).numpy())
print('MSE', mean_squared_error(y_test, pred_test), 'R2', r2_score(y_test, pred_test))
```

실습 ③ 결과 - 같은 답, 다른 방법



—●— 훈련 손실 (MSE, 표준화 단위)

epoch 20 이후로는 0.0057 에서 더 안 내려감 - 잡음(rng.normal 20) 때문에 최대치

	3주차 · sklearn	오늘 · PyTorch
계수 (배터리, 속도)	[12.09, -142.55]	[12.09, -142.55]
절편	-15.74	-15.74
시험 MSE	513.41	513.41
시험 R ²	0.9925	0.9925
predict([[60, 1.2]])	538.8 m	538.8 m

- PyTorch 가 100번 걸어 내려가 찾은 계수가 sklearn 이 공식으로 푼 계수와 소수 둘째 자리까지 같음.
- 즉 `nn.Linear(2, 1) + MSE + 경사하강` = 선형회귀. 딥러닝은 회귀의 연장
- 계수 복원: 표준화 단위의 가중치 $[0.978, -0.241]$ 에 (σ_y / σ_x) 를 곱하면 원래 단위로 돌아옴

표준화를 빼면 - 신경망이 죽는 가장 흔한 방식

343,559

epoch 1 손실 (원 단위, lr=0.01)

2.0×10^9

epoch 2

4.5×10^{20}

epoch 5

nan

그 뒤 - 숫자가 넘쳐 무너진다

왜 죽나

- 배터리는 20~100, 속도는 0.5~2.0, 주행거리는 수백 - 크기가 100배씩 다름
- 기울기도 그만큼 커지고, 한 걸음(lr × 기울기)이 골짜기를 한참 넘어감
- LinearRegression 은 공식으로 풀어서 스케일에 안 흔들렸을 뿐임

처방 - 두 가지 중 하나, 보통은 둘 다 사용

- **X도 y도 StandardScaler**
- lr 을 10분의 1 씩 줄여 본다
- 분류부터는 y가 0/1 이라 X 만 맞추면 됨

실습 ④ 층 하나 더 얹기 - 신경망이 되는 순간

선형층 하나 (실습 ③)

파라미터 3개 = 3주차 선형회귀

```
nn.Sequential(  
    nn.Linear(2, 1)  
)
```

- 시험 R^2 0.9925
- 직선(평면)밖에 못 그린다

숨은층 16 + ReLU (실습 ④)

파라미터 65개 = 다층 퍼셉트론(MLP)

```
nn.Sequential(  
    nn.Linear(2, 16), nn.ReLU(),  
    nn.Linear(16, 1)  
)
```

- 시험 R^2 0.9912 (Adam lr=0.05 · 300 epoch)
- **구부러진 규칙도 배울 수 있다 - 그런데 여기선 안 좋아짐**

- 왜 안 좋아졌나 - 이 데이터의 진짜 규칙(12·배터리 - 150·속도)이 직선이라서. 복잡한 모델이 항상 이기지 않는다
- **그래도 다섯 줄은 한 글자도 안 바뀜. 층을 쌓는 일과 학습하는 일은 분리돼 있다 (이것이 PyTorch 의 설계)**

PyTorch 에서 자주 하는 실수 여섯 가지

① `zero_grad()` 를 뺀다

기울기가 누적돼 엉뚱한 방향으로. 손실이 줄다 말다 함.
루프 첫 줄에 고정

② `dtype` 이 다르다

`int64` · `float64` 데이터를 `float32` 모델에 'expected Float but found Double'.
`dtype=torch.float32`

③ `y` 가 `(n,)` 이다

출력 `(n,1)` 과 안 맞아 손실이 이상하게 계산되거나 경고
`reshape(-1, 1)`

④ 표준화를 안 한다

손실이 수십만 → nan. 신경망이 죽는 1번 원인
`StandardScaler` 먼저

⑤ `lr` 이 크다 / 작다

튀면 10분의 1, 안 움직이면 10배. 곡선을 보고 정한다.
`loss` 를 매 10 epoch 출력

⑥ 예측할 때 `no_grad` 를 뺀다

틀리진 않지만 메모리를 잡아먹고 `.numpy()` 가 막힌다.
`with torch.no_grad():`

에러 문장을 읽는 습관 - 대부분 `dtype` 이나 `shape` 를 그대로 말해줌

암기 - PyTorch 회귀 기본 형태

```
import torch, torch.nn as nn, torch.optim as optim

X_t = torch.tensor(X_train_s, dtype=torch.float32)          # 표준화된 X
y_t = torch.tensor(y_train_s, dtype=torch.float32).reshape(-1, 1) # 표준화된 y, (n,1)

model    = nn.Sequential(nn.Linear(2, 1))                  # ① 모델   (분류·MLP 는 여기만 바뀐다)
loss_fn  = nn.MSELoss()                                     # ② 손실   (분류: nn.CrossEntropyLoss())
opt      = optim.SGD(model.parameters(), lr=0.1)           # ③ 옵티마이저 (또는 optim.Adam(..., lr=0.001))

for epoch in range(100):                                   # ④ 다섯 줄
    opt.zero_grad()
    pred = model(X_t)
    loss = loss_fn(pred, y_t)
    loss.backward()
    opt.step()

with torch.no_grad():                                     # ⑤ 예측 - 기록 끄고
    pred = model(X_new_t)
```



정리

① fit() 은 다섯 줄

지우고 → 예측하고 → 재고 → 미분하고 → 옮긴다.
zero_grad · model(X) · loss_fn ·
backward · step. 순서까지.

② 딥러닝은 이전 회귀의 연장

nn.Linear(2,1) + MSE + 경사하강 = 선형회귀.
계수 [12.09, -142.55] 가 똑같이 나옴. 층을 더
쌓으면 신경망

③ 스케일 · dtype · shape

신경망이 죽는 이유의 대부분. 표준화 먼저, float32,
y는 (n,1). 에러 문장을 읽으면 답이 있다.

지우고 · 예측하고 · 재고 · 미분하고 · 옮긴다

다음 시간에 봅시다.